

Zero-TTT

AlphaZero 与端到端测试时训练融合项目规划

个人研究与娱乐项目

2026 年 8 月

摘要

本文规划一个面向 19 路围棋的 AlphaZero 与 Test-Time Training (TTT) 融合项目。模型使用标准可二阶求导的空间注意力、AlphaGo Zero 风格的八局面输入，以及可在真实落子后更新的快权重记忆。系统只学习一份共享快权重初始化；每盘棋开始时，黑白双方从该初始化派生出两份相互隔离的运行时记忆，随后根据各自搜索结果和观察到的公开落子独立变化。项目以验证机制、完成训练闭环和保证可恢复性为目标，不安排消融实验、超参数搜索或棋力指标分析，也不维护多 batch 下不同样本的 TTT 状态。

棋盘与规则

19×19，贴目 7.5，面积计分，362 个动作

序列批量

固定 batch size 为 1；允许跨序列梯度累积

局部上下文

当前局面加过去 7 个局面，共 8 个局面

长期记忆

一份共享可训练初始化，两份每局私有运行时快权重

二阶窗口

最近 16 个真实回合，整盘保留快权重数值状态

默认执行方式

PyTorch eager mode；暂不使用 `torch.compile`

主要算力目标

Linux + Docker + 单张 NVIDIA GPU；CPU 用于开发测试

目录

1	项目定位与理论判断	4
1.1	项目目标	4
1.2	可行性结论	4
2	围棋状态、动作与规则环境	4
2.1	固定问题定义	4
2.2	网络输入与输出	5
3	模型结构与共享初始化	5
3.1	参考 Transformer	5
3.2	一份可训练初始化，两份运行时状态	6
3.3	信息隔离边界	6
4	真实回合协议	7
5	训练目标与元学习	7
5.1	内循环	7
5.2	外循环	8
6	显存控制、梯度截断与恢复	8
6.1	16 手截断元梯度	8
6.2	Activation Checkpoint	8
6.3	暂不编译模型	8
6.4	持久化检查点	9
7	监督预训练数据	9
7.1	数据来源	9
7.2	预训练顺序	9
8	MCTS 与自博弈设置	10
8.1	搜索基线	10
8.2	模拟次数随机化	10
9	工程组织与容器化	10
10	敏捷开发冲刺	11
11	测试与验收标准	11

11.1 规则与数据正确性	11
11.2 共享初始化与隐私	12
11.3 梯度、截断与恢复	12
12 风险与明确不做的事项	12
13 推荐实施顺序	13

1 项目定位与理论判断

1.1 项目目标

Zero-TTT 的目标不是复现 AlphaGo Zero 的最终棋力，也不是验证 TTT 在围棋上的统计显著收益，而是完成以下工程闭环：

- 在可靠的 19 路围棋规则环境上运行 AlphaZero 风格的策略-价值网络与 MCTS；
- 让模型在一盘棋内通过真实落子持续更新私有快权重，把超过八局面窗口的信息压缩到参数状态中；
- 在训练时对快权重更新求二阶梯度，使共享初始化适合后续在线学习；
- 支持监督预训练、自博弈训练、梯度截断、检查点恢复和 Docker 迁移；
- 保持个人项目所需的可读性，避免多 batch、多设备和复杂分布式状态管理。

1.2 可行性结论

该思路在理论和工程上均可实现。TTT-E2E 将模型权重视为随序列更新的隐状态，并通过外循环元学习优化其初始化[1]；围棋棋局同样是有顺序的长上下文，可以把已经发生的真实决策视为测试时训练样本。

本项目与语言模型 TTT-E2E 并不完全等价，主要差异如下：

- 语言模型使用自然发生的下一 token 作为内循环目标；本项目中，行棋方优先使用 MCTS 根访问分布，观察方只能使用公开的实际动作。
- MCTS 是离散搜索过程，本项目不对 **MCTS 本身求导**。访问分布被视为停止梯度的策略改进目标，因此“端到端”只表示外循环梯度穿过网络和快权重更新。
- 训练时只精确传播最近 16 手的二阶梯度，早期更新通过截断近似处理，所以实现结果不是整盘精确元梯度。

因此，本项目应被描述为“受 TTT-E2E 启发的、带私有在线参数记忆的 AlphaZero”，而不是原论文算法在围棋上的直接移植。是否提升棋力是开放问题，不作为项目完成条件。

2 围棋状态、动作与规则环境

2.1 固定问题定义

项目只支持 19×19 棋盘，动作空间固定为 361 个落点加 1 个停着 (pass)，共 362 个动作。默认贴目为 7.5，采用面积计分。MVP 不支持让子棋、其他贴目、日式数子、其他棋盘尺寸或认输。

规则实现采用固定版本的 PettingZoo go_v5[5]，并在其 Minigo Position 之上封装适合树搜索的不可变状态接口。项目适配层负责：

- 克隆或派生子状态、生成合法动作掩码、提交真实动作；
- 维护完整动作历史和最近八个棋盘局面；
- 在策略输出前屏蔽非法动作；
- 将终局结果统一转换为当前行棋方视角的 $z \in \{-1, +1\}$ 。

2.2 网络输入与输出

输入张量固定为

$$x_t \in \{0, 1\}^{1 \times 17 \times 19 \times 19}.$$

前 16 个平面表示当前及过去 7 个局面中双方棋子的位置，最后一个平面表示当前行棋方。开局不足八个局面时，缺失历史使用全零平面填充。该形式与 AlphaGo Zero 的历史局面表示保持一致[2]。

网络另接收一个非空间角色标记

$$r \in \{\text{self}, \text{opponent}\},$$

用来区分“用自己的搜索分布学习”与“根据对手公开动作学习”。角色标记通过小型嵌入加入全局 token，不改变 17 平面棋盘接口。

输出包括：

- 策略 logits $l_t \in \mathbb{R}^{1 \times 362}$ ，经合法动作掩码后归一化；
- 价值 $v_t \in [-1, 1]^{1 \times 1}$ ，表示当前行棋方的预期终局结果。

所有入口必须断言 batch size 为 1。梯度累积只能顺序处理多个独立序列，不能在一个 batch 中同时保存多组快权重。

3 模型结构与共享初始化

3.1 参考 Transformer

MVP 使用固定的小型参考配置，避免在项目早期引入参数搜索：

配置项	默认值
Transformer 块数	8
隐藏维度	256
注意力头数	8
FFN 隐藏维度	1024
空间 token	361 个交叉点 + 1 个全局 token
注意力	完整、非因果、标准缩放点积注意力
归一化	LayerNorm
dropout	0
快权重块	最后 2 个块

每个普通块由注意力和静态 FFN 构成。最后两个块额外增加自适应 FFN，与静态 FFN 形成双分支：静态分支保存稳定知识，自适应分支承担测试时记忆。注意力、嵌入、位置编码、归一化、输出头和静态 FFN 都属于慢权重；只有自适应 FFN 参与内循环更新。

3.2 一份可训练初始化，两份运行时状态

系统只学习一份快权重初始化

$$\theta_{\text{fast}} = \{\theta_j\}_{j \in \mathcal{F}},$$

其中 $\mathcal{F}$ 是最后两个块内自适应 FFN 的参数集合。不存在独立的可训练黑方初始化 θ^B 或白方初始化 θ^W 。

每盘开始时创建两个数值相同但对象独立的运行时状态：

$$\phi_0^B = \text{clone}(\theta_{\text{fast}}), \quad \phi_0^W = \text{clone}(\theta_{\text{fast}}).$$

这一设计表达了“共享一般围棋知识和学习方式，但根据执黑、执白及实际经历形成不同短期状态”的归纳偏置。它具有两个重要性质：

1. 不增加可训练参数量。优化器和普通模型检查点只维护一份 θ_{fast} 。
2. 不会消除运行时状态成本。一旦 ϕ^B 与 ϕ^W 分化，为保证隐私与独立适应，仍必须保存两份运行时张量。未来可以研究低秩增量以降低该成本，但不属于 MVP。

3.3 信息隔离边界

黑方 MCTS 只能读取 ϕ^B ，白方 MCTS 只能读取 ϕ^W 。一方的根访问分布、搜索树统计和快权重梯度不得作为另一方的更新输入。这里的“隐私”是对局算法内部的信息隔离，并非密码学安全；集中式自博弈进程和离线学习器仍可保存完整轨迹。

4 真实回合协议

设第 t 手的当前行棋方为 c ，另一方为 $\bar{c}$ ，真实落子前状态为 s_t 。

1. 从 ϕ_t^c 创建只读快照，在整个 MCTS 中保持不变；所有假想分支共享该快照，不执行 TTT 更新。
2. MCTS 输出根访问分布 π_t 、根价值和访问次数；按温度规则选择动作 a_t 。
3. 由真实规则环境验证并提交 a_t 。只有提交成功才允许发生快权重更新。
4. 行棋方使用自己的私有搜索分布执行一次内循环更新：

$$\ell_t^{\text{self}} = \text{CE}(\pi_t, p(s_t; \phi_t^c, r = \text{self})) .$$

5. 观察方不得读取 π_t ，只根据公开动作执行对手建模更新：

$$\ell_t^{\text{obs}} = \text{CE}(\text{onehot}(a_t), p(s_t; \phi_t^{\bar{c}}, r = \text{opponent})) .$$

6. 两份快权重的版本号各自递增。由于下一手使用的网络状态已经改变，清空全部旧搜索树，不在 MVP 中复用子树。

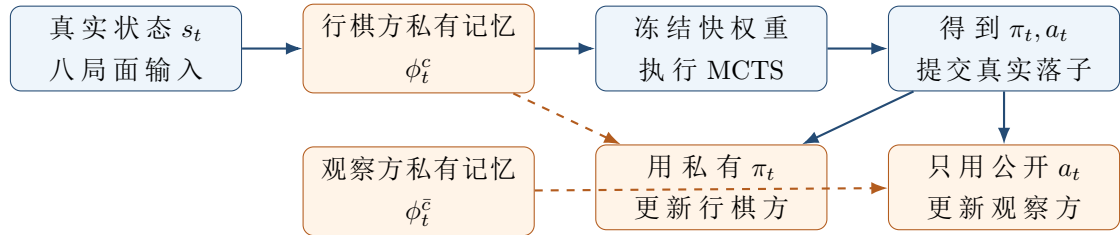


图 1: 一次真实回合中的搜索、提交与双私有记忆更新。虚线表示私有状态流；访问分布不会传给观察方。

5 训练目标与元学习

5.1 内循环

内循环只更新自适应 FFN，使用一次无动量 SGD：

$$\phi_{t+1} = \phi_t - \eta \nabla_{\phi_t} \ell_t(\phi_t) .$$

行棋方使用 ℓ_t^{self} ，观察方使用 ℓ_t^{obs} 。内循环学习率 η 由固定配置给出，MVP 不进行搜索。训练时使用 `torch.autograd.grad` 并设置 `create_graph=True`，从而保留二阶梯度。运行时参数字典则通过 `functional.call` 传入模型[8, 9]。

5.2 外循环

在轨迹的每个真实状态上，只评价当前行棋方的策略和价值：

$$\mathcal{L}_t = w_t \text{CE}(\pi_t, p_t) + \lambda_v (z - v_t)^2 + \lambda_2 \|\theta\|_2^2,$$

其中 z 是终局结果， w_t 是依据实际 MCTS 访问数得到的权重。 π_t 、 z 和动作均停止梯度。观察方更新是否有效，不通过当前观察损失直接评价，而通过该玩家未来成为行棋方时的策略-价值损失获得信用。

外循环只更新共享慢权重和唯一的 θ_{fast} 。一盘棋结束后的 ϕ^B 、 ϕ^W 不写回模型参数；下一条训练序列重新从更新后的共享初始化派生两份状态。

6 显存控制、梯度截断与恢复

6.1 16 手截断元梯度

整盘 19 路围棋可能包含数百次快权重更新。若保留全部二阶计算图，显存会随手数增长，不适合作为默认实现。项目固定使用 16 个真实回合作为元梯度窗口。

在窗口边界保留快权重的数值，但截断旧图，并以直通方式重新连接共享初始化：

$$\phi_{\text{next}} = \theta_{\text{fast}} + \text{stopgrad}(\phi_{\text{carried}} - \theta_{\text{fast}}).$$

该式前向值等于 ϕ_{carried} ，因此整盘记忆不会数值重置；反向只传播最近 16 手，并把边界梯度近似路由到共享初始化。本文档将这一方法明确标记为截断元梯度近似，而非整盘精确二阶求导。

6.2 Activation Checkpoint

每个 Transformer 块使用 PyTorch 的 `checkpoint` 函数，且显式设置 `use_reentrant=False`。非重入实现会记录前向 autograd 图，支持 `autograd.grad` 和嵌套参数结构，适合本项目的高阶梯度[10]。它通过反向时重算前向降低激活显存，因此预期会增加训练时间。

dropout 固定为零，检查点区域不得依赖会在重算时变化的全局状态。实现完成后必须比较开启和关闭 activation checkpoint 时的输出、一阶梯度及二阶梯度。

6.3 暂不编译模型

MVP 固定使用 PyTorch eager mode，不启用 `torch.compile`、TorchScript 或 ONNX。原因不是编译在理论上不可行，而是动态参数字典、`create_graph=True`、变长回合、Python 控制流和检查点重算容易引发 graph break、守卫失效或重复编译[11]，会显著增加调试面。

未来若确有需要，只考虑单独编译无 TTT 更新的纯前向推理函数；启用前必须与 eager mode 做策略、价值、一阶梯度和搜索结果的一致性验证。

6.4 持久化检查点

学习器只在 16 手窗口边界或完整梯度累积边界保存，不序列化 autograd 图。标准训练检查点包含：

- 模型慢权重及唯一的 θ_{fast} ；
- 外循环优化器与学习率调度器状态；
- Python、NumPy、CPU 和 CUDA 随机数状态；
- 数据游标、全局训练步、已完成累积数；
- 配置文件、代码版本和数据清单的哈希。

若需要恢复一盘未结束的自博弈，另存动作历史、八局面缓存、 ϕ^B 、 ϕ^W 、双方记忆版本和随机数状态。MCTS 树不进入检查点；恢复后从当前真实状态重新搜索。若训练在 16 手窗口内部中断，恢复时从最近窗口边界重放最多 16 手。

7 监督预训练数据

7.1 数据来源

监督预训练优先采用 KataGo g104 公共自博弈数据。该数据目录在 CC0 下发布，包含拼接 SGF、训练 NPZ、MCTS 策略访问目标、终局结果和棋局标识[6, 7]。它主要采用面积计分，与 MVP 规则方向一致。

数据下载必须使用显式清单记录来源 URL、文件大小、校验和和许可证；不得把数据打包进 Git 或 Docker 镜像。预处理固定执行：

1. 过滤 19×19、面积计分、普通自博弈样本；
2. 按棋局标识和手数排序，并把不连续位置切分为独立序列；
3. 从连续状态重建项目所需的八局面输入；
4. 优先把 MCTS 访问次数归一化为 π_t ，缺失时退化为实际动作 one-hot；
5. 为整条序列采样同一个八重对称变换，确保棋盘历史、动作、访问分布和合法掩码一致；
6. 逐条序列输出，保持 batch size 为 1。

7.2 预训练顺序

每条监督序列开始时，从同一 θ_{fast} 创建黑白两份私有状态，然后按真实手顺重放。存在搜索分布时，行棋方使用该分布更新；观察方始终只使用公开动作。终局结果用于价值目标。

允许顺序处理 8 条序列后再执行一次外循环优化器更新，以梯度累积改善稳定性。任意时刻内存中只有当前序列的两份运行时快权重，不构造多样本 TTT batch。

8 MCTS 与自博弈设置

8.1 搜索基线

MVP 使用 PUCT。根节点在自博弈中加入比例 0.25、 $\alpha = 0.03$ 的 Dirichlet 噪声。前 30 手按访问分布采样，之后选择访问次数最大的动作。认输关闭，确保每盘棋产生真实终局目标。

搜索过程必须满足：

- 单次搜索内快权重版本固定；
- 非法动作的先验概率为零；
- 叶节点使用当前行棋方的私有记忆求策略与价值；
- 备份价值时正确翻转行棋方视角；
- 真实落子并更新快权重后，丢弃旧树。

8.2 模拟次数随机化

为降低自博弈平均成本并保留部分高质量策略目标，每个真实回合独立选择搜索预算：

- 75% 的回合使用 128 次快速搜索；
- 25% 的回合使用 800 次完整搜索。

两类回合都执行 TTT 更新。外循环策略损失按实际访问数相对完整预算加权，避免少量访问产生的尖锐分布与完整搜索拥有相同监督强度。该思路参考 KataGo 的 `playout cap randomization`^[4]，但 MVP 不加入其余复杂搜索启发式。

9 工程组织与容器化

建议实现阶段采用以下模块边界：

子系统	职责
规则适配层	不可变围棋状态、合法动作、八局面历史、视角转换与终局结果
模型层	Transformer、策略/价值头、共享快权重初始化与函数式前向
记忆控制器	创建黑白状态、角色化内循环更新、版本管理与 16 手重锚定
MCTS	PUCT、根噪声、随机预算、访问分布和树生命周期

数据层	g104 下载清单、连续序列重建、对称增强与 batch=1 流式读取
学习器	监督预训练、外循环损失、梯度累积、activation checkpoint 和恢复
自博弈器	双私有记忆回合协议、轨迹写入与未完成棋局快照

Docker 以 Linux、Python 3.12、PyTorch 2.13、CUDA 12.8 和单张 NVIDIA GPU 为参考环境。数据、日志和检查点全部通过 volume 挂载。容器至少提供以下独立命令入口：规则测试、数据预处理、监督预训练、自博弈、AlphaZero 学习器和检查点恢复。

本地 CPU 路径只要求能运行规则测试、数据测试、小模型前向、一阶梯度和小规模二阶梯度，不承诺完成 19 路训练。

10 敏捷开发冲刺

冲刺	主题	完成定义
0	规则与可微性	完成 19 路规则适配、17 平面编码、共享初始化、双运行时状态、标准注意力双反向和非重入 activation checkpoint 最小验证。
1	序列数据	完成 g104 数据清单、许可证记录、连续序列重建、整序列对称增强、batch=1 加载和梯度累积。
2	标准 AlphaZero	完成策略-价值网络、PUCT、根噪声、搜索预算随机化、自博弈轨迹和一盘完整 19 路对局。
3	私有 TTT	完成唯一共享初始化、黑白状态分化、行棋方私有目标、观察方公开动作目标、16 手截断元梯度和树缓存失效。
4	恢复与容器化	完成学习器检查点、未完成棋局恢复、Docker CPU/GPU 命令及端到端冒烟测试。
未来	非 MVP	前端、后端、棋盘可视化、多 GPU、batch> 1、模型编译、低秩快权重、消融实验、参数搜索和 Elo/性能分析。

11 测试与验收标准

11.1 规则与数据正确性

- 固定棋谱覆盖单子与多子提取、打劫、禁入点、停着、连续双停和终局计分；
- 动作编号、SGF 坐标、策略索引和八重对称变换可逆；

- 八局面输入在提子、停着和开局补零时正确；
- 所有训练样本 batch 维度严格等于 1；同一序列只使用一个对称变换。

11.2 共享初始化与隐私

- 模型参数和普通检查点中只有一份 θ_{fast} ；
- 每盘开始时 ϕ_0^B 与 ϕ_0^W 数值相同，但存储和后续更新相互独立；
- MCTS 前后两份快权重均不变化；真实落子后双方各更新一次；
- 观察方更新结果只依赖公开状态和实际动作，改变对方私有 π_t 不得影响观察方更新；
- 更新后旧树为空，快权重版本与搜索节点版本不存在混用。

11.3 梯度、截断与恢复

- 快权重更新对 θ_{fast} 产生有限、非零的二阶梯度；
- activation checkpoint 开关前后的输出、一阶梯度和二阶梯度在规定容差内一致；
- 16 手边界前后快权重前向数值连续，旧 autograd 图已释放；
- 可完成 batch=1 监督更新和 8 序列梯度累积；
- 学习器恢复后下一步结果与无中断运行一致；未完成棋局恢复后会清空并重建 MCTS 树；
- Docker CPU 冒烟测试和单卡 CUDA 冒烟测试通过。

项目最终以机制正确、训练闭环可运行、检查点可恢复和设备可迁移为完成标准，不以 Elo、吞吐率、训练时长或相对基线提升作为验收指标。

12 风险与明确不做的事项

- **二阶显存仍可能过高**：先缩小调试模型验证正确性；正式配置使用 16 手截断与逐块 activation checkpoint，不在 MVP 中改成一阶近似。
- **观察方更新可能带来目标冲突**：通过 memory_role 显式区分自身策略和对手建模；不通过新增消融实验解决。
- **公开数据未必形成完整连续序列**：按棋局与手数切分，只接受连续片段；策略访问目标缺失时使用实际动作，不凭空插值。
- **MCTS 缓存失效增加成本**：MVP 选择正确性优先，每手清树；带版本重评估的树复用留到未来。
- **共享初始化不等于共享运行时记忆**：参数量只计算一份初始化，但对局中仍保留两份已经分化的状态，文档和实现不得混淆两者。

明确不属于当前范围的内容包括：多 batch 快权重管理、分布式多 GPU、自动超参数搜索、消融实验、性能基准、棋力评级、前端、后端服务、棋盘可视化、模型编译和低秩快权重压缩。

13 推荐实施顺序

首先完成小模型的共享初始化、双状态隔离和二阶梯度测试，再接入完整 19 路 Transformer。随后打通规则与 MCTS，但暂不启用 TTT；确认标准 AlphaZero 回合正确后，再加入真实落子后的两类私有更新。最后接入监督序列、16 手截断、持久化检查点和 Docker。

这一顺序把最高风险的三个问题——规则正确性、快权重信息边界和高阶梯度可用性——放在项目早期验证，同时保留一个始终可运行的标准 AlphaZero 基线。基线用于工程回退和故障定位，不用于正式消融或性能比较。

参考文献

- [1] Arnub Tandon, Karan Dalal, Xinhao Li, et al. *End-to-End Test-Time Training for Long Context*. 项目本地论文: [paper/End-to-End Test-Time Training for Long Context.pdf](#).
- [2] David Silver, Julian Schrittwieser, Karen Simonyan, et al. Mastering the Game of Go without Human Knowledge. *Nature*, 550:354–359, 2017. <https://www.nature.com/articles/nature24270>
- [3] David Silver, Thomas Hubert, Julian Schrittwieser, et al. A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play. *Science*, 362:1140–1144, 2018.
- [4] David J. Wu. Accelerating Self-Play Learning in Go. arXiv:1902.10565, 2020 revision. <https://arxiv.org/abs/1902.10565>
- [5] Farama Foundation. PettingZoo Go Environment, go_v5. https://pettingzoo.farama.org/main/_modules/pettingzoo/classic/go/go/
- [6] KataGo Project. g104 Downloadable Self-Play Data: Format and Contents. <https://katagoarchive.org/g104/README.txt>
- [7] KataGo Project and Jane Street. g104 Data License (CC0). <https://katagoarchive.org/g104/LICENSE.txt>
- [8] PyTorch Contributors. `torch.func.functional_call` Documentation. https://docs.pytorch.org/docs/main/generated/torch.func.functional_call.html
- [9] PyTorch Contributors. `torch.autograd.grad` Documentation. <https://docs.pytorch.org/docs/stable/generated/torch.autograd.grad.html>
- [10] PyTorch Contributors. Activation Checkpointing Documentation. <https://docs.pytorch.org/docs/stable/checkpoint.html>
- [11] PyTorch Contributors. `torch.compile` Troubleshooting and Graph Breaks. https://docs.pytorch.org/docs/main/user_guide/torch_compiler/torch.compiler_troubleshooting.html